

EMBEDDED SYSTEM WITH IOT

EXPERIMENT NO. 3

IOT BASED WEATHER MONITORING SYSTEM

Name: *Raphael S. Nano*

Course/Section: *CPE162P / CI*

Date of Performance: *March 4-6, 2024*

Date of Submission: *March 8, 2024*

Ma'am Cyrel Manlises

Instructor

DISCUSSION

Weather monitoring systems based on IoT are widely used in mobile embedded systems. These systems serve as a hub for meteorological knowledge by continuously monitoring and predicting weather patterns, collecting and analyzing information with speed and precision. They offer a wide variety of analysis for weather reports using a network of sensors and grounded measurement tools. With that said, an IoT-based Weather Monitoring System with detection and output operations was constructed through the use of a DHT11 temperature and humidity sensor, a liquid crystal display, and an ESP8266 Wi-Fi module.

For experiment 3, the objectives are as follows:

- 1. To demonstrate how an IoT based weather monitoring system works.*

2. To be able apply what you have learned in logic circuits and switching theory, microprocessors, CPE160P, and CPE161P.

3. To be able to apply embedded system with IoT using Arduino kit

4. To be able to use ThingSpeak for the data collection and analysis.

The materials involved in constructing the IoT-based Weather Monitoring System are the ESP8266 WiFi module, DHT11 temperature and humidity sensor, and liquid crystal display.

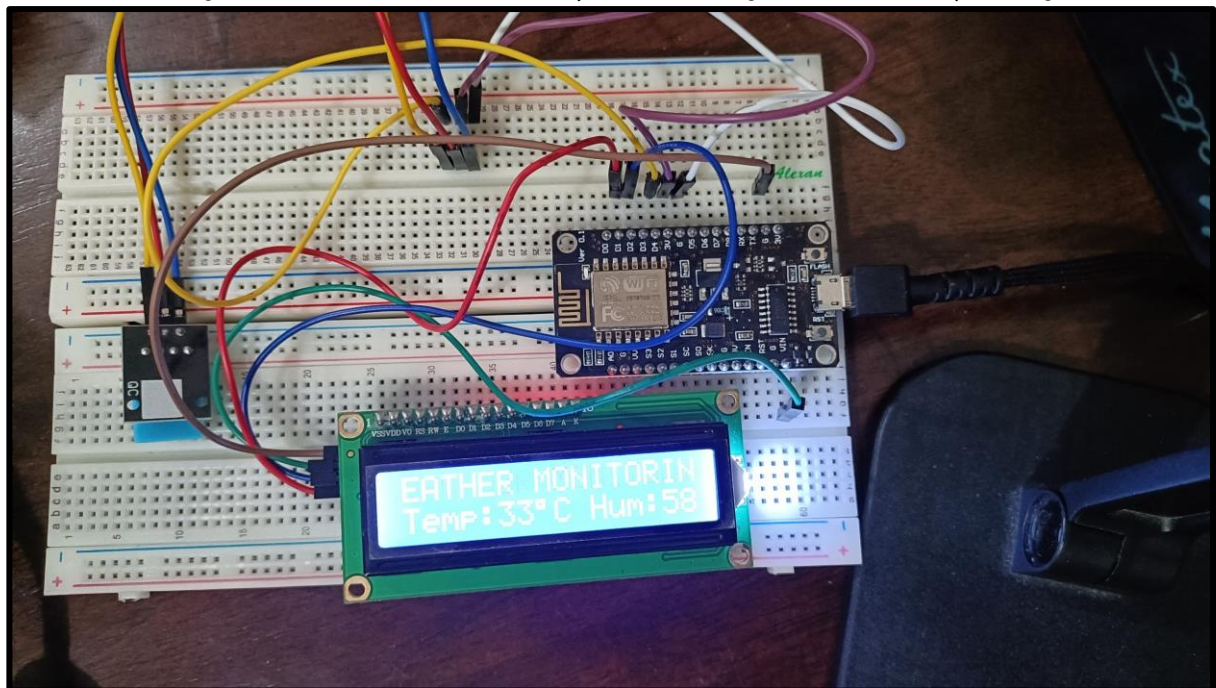


Figure 1. Major Components for the Experiment

For the external libraries, ESP8266WiFi.h and ThingSpeak.h are some of the separate libraries included. Secrets.h is a file with definitions for ESP8266WiFi.h. For the LCD, LiquidCrystal_I2C.h was utilized, and dht.h for DHT11.

```
#include <ESP8266WiFi.h>
#include "ThingSpeak.h"
#include <Arduino.h>

#include <dht.h>
#include <LiquidCrystal_I2C.h>
#include <String.h>
```

Figure 2. External Libraries Included

The primary aim of experiment 3 is to build an IoT based weather monitoring system that streams temperature and humidity data over the internet to the user. Moreover, for the collection of data and analysis, ThingSpeak will be used for graphing of values.

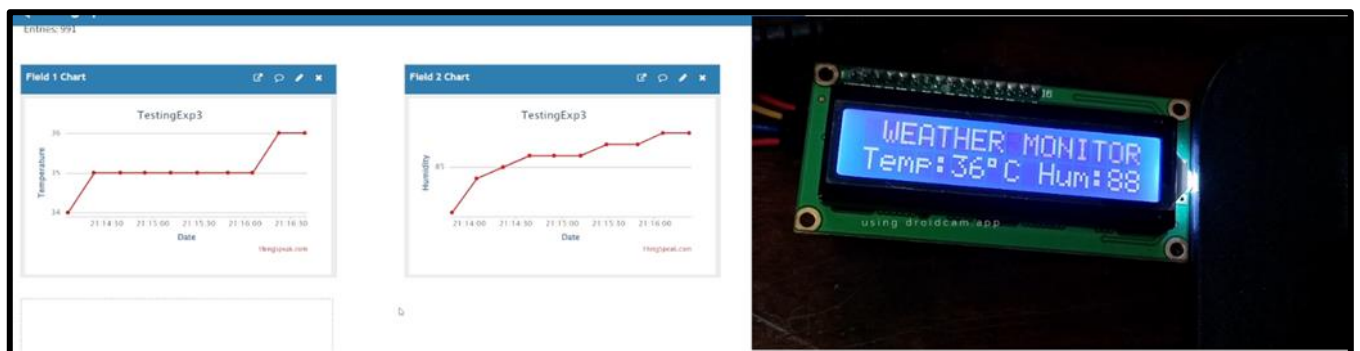


Figure 3. Sample Out put of the Experiment

The main components of this experiment are the detection of temperature and humidity and the displaying of values in the LCD. With the use of read11() function, the DHT detects the temperature and humidity. The lcd.print() function is then utilized in order to output the value of the DHT.

```
DHT.read11(2);  
Serial.println(DHT.temperature);  
lcd.setCursor(0, 1);  
lcd.print("Temp:");  
lcd.setCursor(5, 1);  
lcd.print(int(DHT.temperature));  
lcd.print((char)223);  
lcd.print("C");  
lcd.setCursor(10, 1);  
lcd.print("Hum:");  
lcd.setCursor(14, 1);  
lcd.print(int(DHT.humidity));
```

Figure 4. DHT11 Sensor Main Function

A WiFi connection was first established in order to be able to connect the ESP8266 and the ThingSpeak. A code was again created, wherein it connect to a certain network via Wi-Fi (fh_dbe600) and makes multiple attempts until an adequate connection is established. It provides

information to the user via the Serial Monitor, indicating the status of the connection attempt.

```
if (WiFi.status() != WL_CONNECTED) {  
  Serial.print("Attempting to connect to SSID: ");  
  Serial.println("fh_dbe600");  
  while (WiFi.status() != WL_CONNECTED) {  
    WiFi.begin(ssid, pass);  
    Serial.print(".");  
    delay(5000);  
  }  
  Serial.println("\nConnected.");  
}
```

Figure 5. WiFi Function

For the updating of the fields in ThingSpeak, it occurs every 15 seconds. In this case, there are two fields: one for temperature and one for humidity. The updating of the values is done using the writeFields function of the ThingSpeak.

```
if ((millis() - timeStart) >= 15000) {  
  ThingSpeak.setField(1, int(DHT.temperature));  
  ThingSpeak.setField(2, int(DHT.humidity));  
  
  int x = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);  
  if (x == 200) {  
    Serial.println("Channel update successful.");  
  } else {  
    Serial.println("Problem updating channel. HTTP error code " + String(x));  
  }  
  timeStart = millis();  
}
```

Figure 6. Thingspeak Function

In the main loop, the detection of the temperature and humidity using DHT11 is

continuous. On the other hand, the updating of the output of the current value of temperature and humidity in ThingSpeak is every 15 seconds.

```
void loop() {
  if ((millis() - timeStart) >= 15000) {
    ThingSpeak.setField(1, int(DHT.temperature));
    ThingSpeak.setField(2, int(DHT.humidity));

    int x = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);
    if (x == 200) {
      Serial.println("Channel update successful.");
    } else {
      Serial.println("Problem updating channel. HTTP error code " + String(x));
    }
    timeStart = millis();
  }

  DHT.read11(2);
  Serial.println(DHT.temperature);
  lcd.setCursor(0, 1);
  lcd.print("Temp:");
  lcd.setCursor(5, 1);
  lcd.print(int(DHT.temperature));
  lcd.print((char)223);
  lcd.print("C");
  lcd.setCursor(10, 1);
  lcd.print("Hum:");
  lcd.setCursor(14, 1);
  lcd.print(int(DHT.humidity));
  scrollText(0, title, 350, 16);
}
```

Figure 7. Main Loop Function

RESULTS

The IoT-based Weather Monitoring System consisted of various conditions in order to fully test the system. For this experiment, there are 10 conditions: Room temperature in the (1) night and in the (2) day, (3) air-conditioned room temperature, (4) hand, comfort room of (5) dorm and (6) school, (7) bag, (8) cabinet, (9) near the window, and (10) school corridor. The temperature and humidity of these places or objects were measured using DHT11, and the value was also posted in ThingSpeak. As seen below in the graphs, most of the conditions (i.e., room temperatures, comfort rooms, cabinet) have a temperature of 30 to 31 degrees Celsius while the humidity ranges from 50 to 60. For enclosed conditions such as in bags and hands, the temperature increases to 33 - 35 degrees Celsius and the humidity also increases to 60 above. Lastly, the air-conditioned room displays that the temperature reaches 17 degrees Celsius with a humidity of 30 to 40. This

means that the system is successfully updating the values of the sensor and is simultaneous with the output on the LCD.

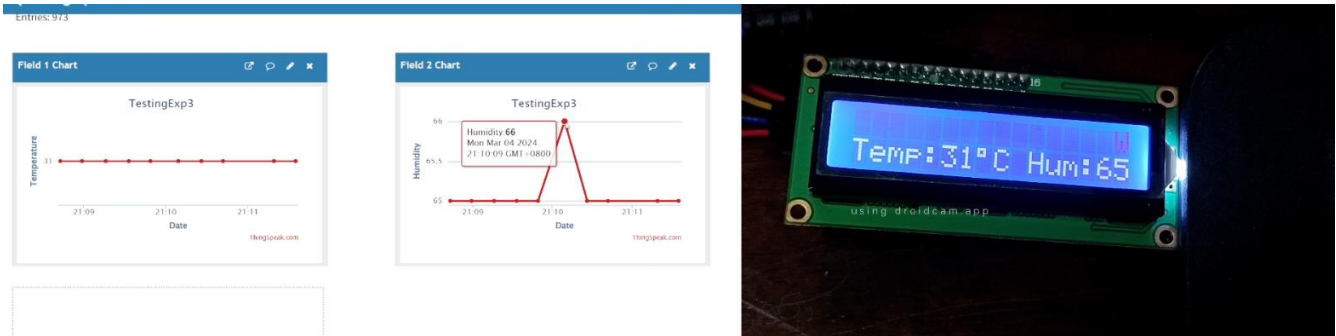


Figure 8. Room Temperature at Day

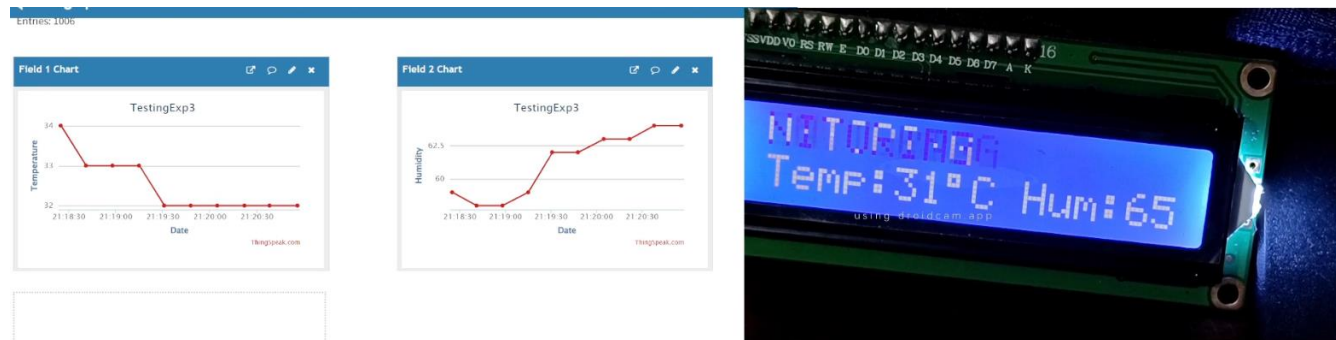


Figure 9. Bag

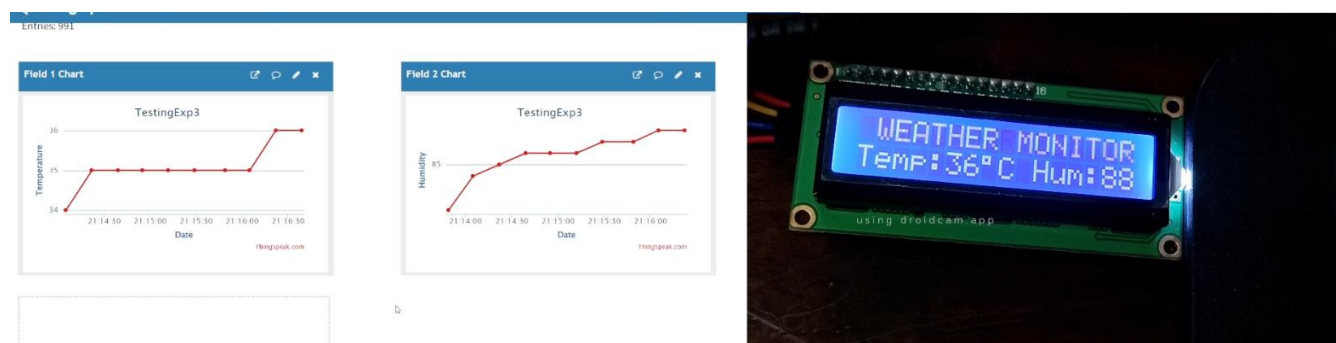


Figure 10. Hand

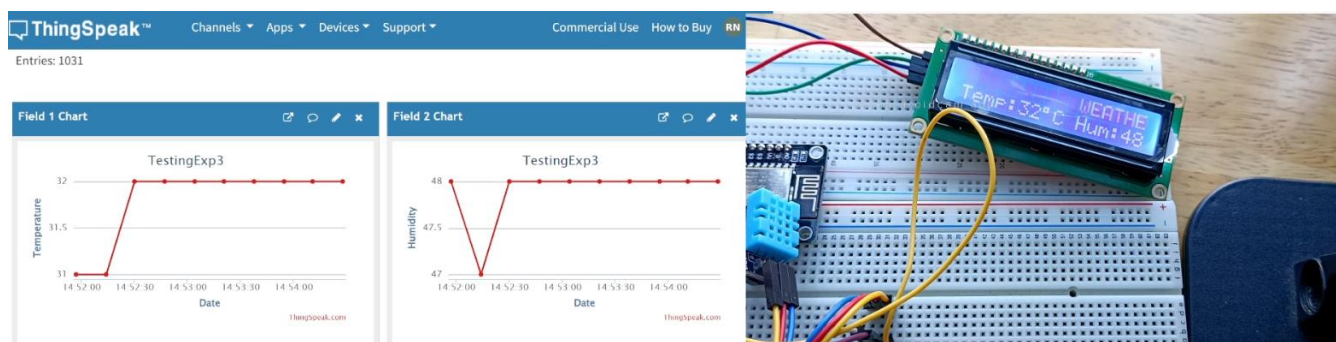


Figure 11. Window

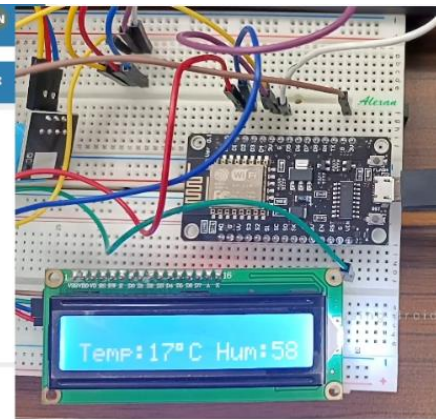
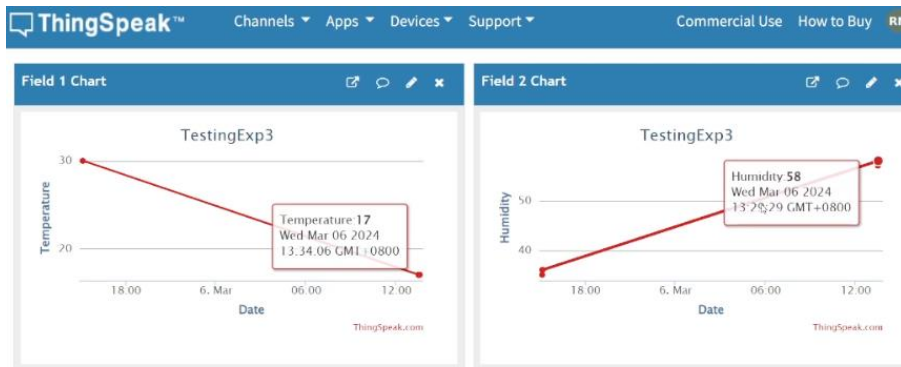


Figure 12. Air-conditioned Room



Figure 13. School Corridors

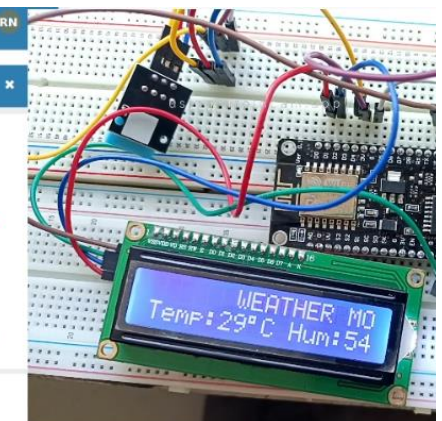


Figure 14. School Comfort Room

CONCLUSION

In conclusion, the IoT-based Weather Monitoring System functions by utilizing the `readIIC()` and `Icd.print()` function, wherein the temperature and humidity is printed on the LCD. Every 15 seconds, the ThingSpeak is being updated, wherein both the temperature and humidity values are displayed. The CPE160P and CPE161P lessons were specifically applied by using the Arduino IDE codes in order to connect the ESP8266 with ThingSpeak and applying the libraries for the sensor, thereby achieving the second objective. The ThingSpeak successfully displays the temperature and humidity values every 15 seconds, which was adequately performed by using the `writeFields()` function. Therefore, the IoT-based Weather Monitoring System was successfully created, wherein the values are accurate and the outputs are understandable both in the LCD and in ThingSpeak.